Dokumentasi Kode — Firmware Robot `Finalnew.ino`

Dokumen ini menjelaskan logika firmware `Finalnew.ino` yang berjalan di **ESP32** sisi *Body Robot*. Firmware menerima perintah gerak dari Pengendali Utama (Raspberry Pi) melalui modul radio **NRF24L01**, menjalankan satu perintah dalam satu waktu, lalu melaporkan hasilnya kembali.

> **Acuan perilaku:** *Dokumen Spesifikasi Teknis — Robot Akuatik untuk Tambak Ikan*, bab **Skema Komunikasi**. Berkas `.docx`-nya ada di folder yang sama.

---

1. Gambaran Umum

Firmware ini adalah **pelaksana perintah**, bukan perencana misi. Ia menerima perintah tingkat rendah seperti *"maju 200 cm"* atau *"putar kanan 90 derajat"*, mengeksekusinya, lalu melapor selesai.

Perubahan peran dari versi sebelumnya. Dulu robot menerima satu paket misi (`P=10;L=4;MODE=AUTO`) lalu menjalankan **seluruh pola zigzag sendiri** di dalam firmware. Sekarang pola jalur disusun dan dikemudikan **Pengendali Utama**; robot hanya mengeksekusi satu perintah dan melapor lewat `ON` / `FIN`.

Konsekuensinya:

- Firmware jadi jauh lebih sederhana — tidak ada lagi penghitungan lane, durasi lintasan, atau pembalikan arah U-turn.
- Kemajuan misi yang ditampilkan operator adalah **konfirmasi nyata dari robot**, bukan perkiraan berbasis penghitung waktu.
- Mengubah pola jalur cukup dilakukan di aplikasi, **tanpa flash ulang** ESP32.

Alur kerja singkat:

1. Robot menunggu paket berformat `[Moves]:[Dist]`.
2. Paket divalidasi terhadap tabel perintah (rentang `[Dist]` harus sah).
3. Robot mengirim `ON [Moves]` **seketika**, lalu mengulanginya tiap 1 detik selama bergerak.
4. Gerakan dijalankan: `FW` berbasis waktu tempuh dengan *heading-hold*, `CW`/`CCW` berbasis umpan balik sudut yaw MPU.
5. Setelah tuntas, robot mengirim `FIN [Moves]:[Dist]`.
6. Selama bergerak robot memantau **heartbeat**. Bila sinyal hilang > 3 detik atau menerima `STOP`, robot langsung berhenti (*failsafe*) dan **tidak** mengirim `FIN`.

Sifat penting yang membentuk desain kode:

- **Komunikasi dua arah half-duplex ("gantian").** Pengendali Utama mengirim perintah/PING; robot menerima, lalu bergiliran mengirim balik `ON`/`FIN`/telemetri MPU di sela-selanya — seperti radio HT.
- **Jarak berbasis waktu (open-loop), sudut berbasis sensor (closed-loop).** Robot tidak punya GPS/encoder, jadi *"maju 200 cm"* diwujudkan sebagai *"maju selama sekian milidetik"*. Sebaliknya *"putar 90 derajat"* ditutup memakai pembacaan yaw MPU sehingga **tidak** bergantung kalibrasi durasi.
- **Belok differential thrust.** Tidak ada kemudi; belok dilakukan dengan mematikan satu motor. Robot **berputar melingkar lebar**, bukan di tempat — sudut akhirnya akurat, tapi posisinya ikut bergeser.

---

2. Perangkat Keras & Pustaka

Pustaka yang di-*include*:

Pustaka	Fungsi
`SPI.h`	Bus SPI untuk komunikasi ke modul NRF24L01
`RF24.h`	Driver modul radio NRF24L01
`Wire.h`	Bus I2C untuk sensor MPU6050
`MPU6050_tockn.h`	Pembacaan sudut (roll/pitch/yaw) dari MPU6050
`ESP32Servo.h`	Menghasilkan sinyal PWM servo 50 Hz untuk ESC brushless
`math.h`	Fungsi `fabs()`

Pemetaan pin:

Modul	Pin ESP32	Catatan
NRF24L01 CE / CSN	GPIO4 / GPIO5	**Jangan** pakai GPIO9/10 — pin flash internal WROOM, bikin boot-loop
MPU6050 SDA / SCL	GPIO21 / GPIO22	`Wire.begin(21, 22)`
ESC Kiri / Kanan	GPIO25 / GPIO26	Sinyal servo 50 Hz, rentang 1000–2000 µs

Alamat radio:

Alamat	Arah	Isi
`00001`	Pi → Robot	Perintah gerak & PING
`00002`	Robot → Pi	`ON`, `FIN`, dan telemetri MPU

---

3. Konstanta Kalibrasi

Konstanta	Nilai	Arti
`ROBOT_SPEED_MPS`	`0.5f`	Kecepatan jelajah pada `CRUISE_PWM` (m/s). **Wajib diukur di air** — menentukan ketepatan jarak `FW`
`CRUISE_PWM`	`1300`	Throttle jelajah (µs)
`KP_HEADING`	`8.0f`	Gain proporsional koreksi arah saat maju lurus
`MAX_CORRECTION`	`60`	Batas koreksi PWM (µs) agar tidak ekstrem
`HEADING_DEADBAND`	`2.0f`	Toleransi error arah (derajat) sebelum koreksi aktif
`TURN_TOLERANCE`	`3.0f`	Toleransi sudut akhir putaran (derajat)
`TURN_RATE_DPS`	`60.0f`	Perkiraan laju putar (derajat/detik) — hanya untuk **batas waktu**, bukan penentu sudut
`SIGNAL_TIMEOUT_MS`	`3000`	Failsafe: berhenti bila tak ada sinyal selama ini
`ON_INTERVAL_MS`	`1000`	Interval kirim `ON [Moves]` — **ditetapkan spesifikasi**, jangan diubah
`TELEMETRY_INTERVAL_MS`	`200`	Pembatas laju kirim telemetri MPU
`YAW_SIGN`	`+1`	Arah tanda yaw MPU. **Ubah jadi `-1` bila hasil uji menunjukkan `CW` malah berputar ke kiri**

Batas `[Dist]` — nilainya sama persis dengan tabel perintah pada dokumen spesifikasi:

Konstanta	Nilai
`FW_MIN_CM` / `FW_MAX_CM`	`10` / `200`

Konstanta	Nilai
`TURN_MIN_DEG` / `TURN_MAX_DEG`	`1` / `180`

`TURN_90_MS` dari versi lama **sudah dihapus**. Putaran kini ditutup dengan sensor, sehingga konstanta durasi belok yang rawan itu tidak diperlukan lagi.

---

4. Variabel Status Global

Variabel	Peran
`lastSignalMs`	Waktu paket terakhir diterima — dasar perhitungan failsafe
`stopRequested`	Ditandai `true` saat paket `STOP` masuk di tengah gerakan
`lastTelemetryMs`	Pembatas laju telemetri MPU
`lastOnMs`	Waktu terakhir `ON [Moves]` dikirim
`moveInProgress`	`true` selama sebuah gerakan berjalan
`currentMove`	Nama perintah yang sedang dijalankan (`"FW"` / `"CW"` / `"CCW"`)

Enum `MoveResult` menyatakan alasan berhentinya gerakan:

Nilai	Arti
`MOVE_DONE`	Gerakan tuntas normal → kirim `FIN`
`MOVE_STOPPED`	Dibatalkan perintah `STOP` → **tidak** kirim `FIN`
`MOVE_LOST_SIGNAL`	Sinyal hilang (failsafe) → **tidak** kirim `FIN`

Enum ini sengaja dideklarasikan **di bagian atas berkas**, sebelum fungsi mana pun. Arduino menyisipkan prototipe fungsi otomatis di awal berkas `.ino`; bila enum dideklarasikan di tengah, kompilasi gagal dengan `'MoveResult' does not name a type`.

4b. Kesehatan MPU — `mpuProbe()`, `initMPU()`, `mpuUpdate()`

Seluruh kemudi robot bergantung pada yaw MPU: `FW` memakainya untuk *heading-hold*, `CW`/`CCW` memakainya untuk menutup putaran. Karena itu **MPU yang mati harus diketahui, bukan dipercaya diam-diam**.

Sidik jari MPU yang tidak terbaca

Pustaka `MPU6050_tockn` **tidak memeriksa ACK** sama sekali — `mpu.begin()` tetap terlihat berhasil walaupun tidak ada sensor di bus. Bila I2C tidak menjawab, `Wire.read()` mengembalikan `-1` untuk **setiap** byte, sehingga di dalam `update()`:

```
rawAccX = rawAccY = rawAccZ = (-1 << 8) | -1 = -1
accX = accY = accZ = -1 / 16384 = -6,1e-5

angleAccX = atan2(accY, accZ + abs(accX)) = atan2(-6,1e-5, 0) = -90,0°  PAS
angleAccY = atan2(accX, accZ + abs(accY)) / -1                 = +90,0°  PAS
angleZ    = integral gyro yang habis dikurangi offset           = -0,0°
```

Filter komplementer lalu menarik `angleX`/`angleY` ke nilai akselerometer itu, jadi sudut **terkunci**:

`Roll=-90.0 Pitch=90.0 Yaw=-0.0` yang **tidak pernah berubah walau robot digoyang** bukan pembacaan sensor — itu tanda bus I2C diam. Penyebabnya kabel/daya (SDA=21, SCL=22, 3.3 V, GND, AD0→GND), bukan kalibrasi.

Cara firmware mendeteksinya

Fungsi	Tugas
`i2cScanReport()`	Sapu seluruh alamat `0x01`–`0x7E` saat boot dan cetak apa yang benar-benar menjawab — memberi **fakta**, bukan tebakan
`mpuWhoAmI()`	Baca register `0x75`; MPU6050 asli menjawab `0x68`
`mpuProbe()`	`Wire.endTransmission() == 0` — benar-benar ada yang meng-ACK alamat `0x68`?
`initMPU(calibrate)`	Probe dulu, baru `mpu.begin()`. `calibrate=false` dipakai saat sambung ulang **selagi robot bergerak**, karena kalibrasi menuntut sensor diam
`mpuUpdate()`	Pengganti `mpu.update()` — **semua** pembacaan MPU lewat sini
`maybeCalibrateWhenIdle()`	Menyusulkan `calcGyroOffsets()` bila MPU baru terdeteksi setelah boot

Kalau bus kosong tapi ada `0x69`, itu berarti pin `AD0` modul ter-HIGH. Firmware hanya bicara ke `0x68`, jadi sensor tak akan pernah terbaca — sambungkan `AD0` ke `GND`.

`mpuUpdate()` menilai dua hal tiap pembacaan:

1. ketiga sumbu akselerometer tepat `-1` → tidak ada byte yang terbaca;
2. besar vektor percepatan `|a| = √(ax² + ay² + az²)` harus berada di **0,30 – 3,00 g** — akselerometer sehat selalu merasakan gravitasi ~1 g pada posisi apa pun, sedangkan sensor mati menghasilkan ~0.

Hasilnya disimpan di `mpuOk`, dan setiap perubahan status dicetak ke Serial. Saat `mpuOk` salah, bus di-`Wire.begin()` ulang tiap `MPU_RETRY_INTERVAL_MS` (2 detik) untuk mencoba pulih sendiri.

Gejala kedua: yaw merayap pelan dengan laju tetap

Berbeda dari sudut yang beku, **yaw yang bertambah pelan-pelan (mis. ~0,1°/detik) padahal robot diam** berarti sensor terbaca tetapi **offset giroskopnya nol** — `calcGyroOffsets()` tidak pernah dijalankan.

Penyebabnya: `MPU6050 mpu(Wire)` adalah objek global, sehingga `gyroXoffset`/`gyroYoffset`/`gyroZoffset` ter-inisialisasi `0.0`. Bila probe gagal saat boot, `initMPU(true)` tidak pernah terpanggil; kalau sensor baru muncul belakangan dan hanya disambungkan dengan `calibrate=false`, offsetnya tetap nol selamanya. Bias giroskop mentah lalu diintegrasikan terus-menerus dan yaw merayap.

Karena itu `mpuCalibrated` dilacak terpisah dari `mpuOk`, dan `maybeCalibrateWhenIdle()` menyusulkan kalibrasi begitu robot menganggur. Kalibrasi dipanggil sebagai `calcGyroOffsets(true, 0, 0)` — argumen `delayBefore`/`delayAfter` bawaan pustaka (1000 ms + 3000 ms) sengaja dinolkan karena murni kosmetik, sementara 4 detik diam membuat robot berhenti melayani radio.

Akibatnya pada gerakan

Perintah	Saat `mpuOk` benar	Saat `mpuOk` salah
`FW`	*Heading-hold* proporsional aktif	Koreksi **dimatikan** — maju lurus dua motor sama besar, lebih baik daripada dikemudikan sudut palsu
`CW` / `CCW`	Putaran ditutup umpan balik yaw	Turun ke **open-loop berbasis waktu** (`deg / TURN_RATE_DPS`)

Sebelum ada penanganan ini, putaran dengan MPU mati menunggu target yaw yang tak akan pernah tercapai, sehingga selalu habis di batas waktu: `CCW:90` berjalan `90/60 × 3000 + 3000` = **7500 ms**, padahal seharusnya ~1500 ms — robot berputar sekitar 5× lipat pada setiap belokan.

Status ini juga dikirim ke Pengendali Utama lewat bidang `S` pada telemetri (lihat bagian 14), sehingga GUI dapat menampilkan angka merah dan peringatan alih-alih membiarkan operator membaca sudut beku

sebagai data sungguhan.

---

5. Fungsi Gerak Dasar (Motor Primitives)

Fungsi	Aksi
`armESC()`	Kirim 1000 µs ke kedua ESC lalu tunggu 5 detik — prosedur *arming* wajib ESC brushless
`stopBoat()`	Kedua ESC ke 1000 µs (diam)
`spinCW()`	Motor kiri `CRUISE_PWM`, kanan diam → berputar searah jarum jam (ke kanan)
`spinCCW()`	Motor kiri diam, kanan `CRUISE_PWM` → berlawanan jarum jam (ke kiri)

Karena ESC bersifat **unidirectional** (hanya maju), putaran hanya bisa dilakukan dengan differential thrust. Akibatnya robot berputar sambil bergerak maju melingkar — bukan berputar di tempat.

---

6. Komunikasi Balik — `sendPacket()` dan turunannya

Semua kiriman dari robot ke Pi melewati satu fungsi:

```
void sendPacket(const char *msg)
{
  radio.stopListening();
  radio.openWritingPipe(telemetryAddress);   // alamat balik "00002"
  radio.write(msg, strlen(msg) + 1, true);   // multicast=true -> NOACK
  radio.startListening();                    // langsung dengar lagi
}
```

Window TX-nya sangat pendek dan dikirim **NOACK** (*fire-and-forget*) supaya robot tidak nge-*block* menunggu balasan dan tidak melewatkan perintah berikutnya.

Tiga pembungkusnya:

Fungsi	Kiriman	Kapan
`sendOn(move)`	`ON <move>`	Saat gerakan dimulai + 3 ulangan rapat (120 ms), lalu tiap 1 detik
`sendFin(move, dist)`	`FIN <move>:<dist>`	Saat gerakan tuntas, lalu **diulang** hingga 6× tiap 150 ms
`maybeResendFin()`	ulangan `FIN`	Berhenti begitu perintah gerak berikutnya datang
`maybeSendTelemetry()`	`R=..;P=..;Y=..;S=<kode>`	Setelah menerima paket, dibatasi tiap 200 ms

`maybeSendOn()` memeriksa apakah sudah lewat `ON_INTERVAL_MS` sejak kiriman terakhir; kalau ya, kirim lagi.

Kenapa `FIN` diulang? Paket balik dikirim **NOACK** dan radionya half-duplex. `ON` sudah punya jaring pengaman karena diulang tiap 1 detik, tetapi `FIN` dulu dikirim **sekali saja**. Bila paket itu lepas tepat saat Pengendali Utama sedang giliran mengirim, paketnya hilang selamanya: gerakan yang **sudah tuntas** terlihat "tidak direspon" dan seluruh sisa antrean misi dibatalkan. Pengulangan dihentikan oleh `clearPendingFin()` begitu perintah gerak berikutnya tiba — itu bukti Pengendali Utama sudah menerimanya. Di sisi Pi, tiap `FIN` diberi **cap waktu terima** dan hanya yang tiba **sesudah** perintah dikirim yang dihitung; tanpa itu, dua perintah sejenis berturut-turut (mis. `FW:200` lalu `FW:200`) bisa membuat perintah kedua dianggap selesai oleh kiriman ulang milik perintah pertama.

Kenapa `ON` pertama dikirim seketika? Pengendali Utama hanya menunggu **1 detik** sebelum menganggap perintah gagal dan mengirimnya ulang. Kalau robot menunggu satu siklus penuh dulu, hampir setiap perintah akan terkirim ganda.

Kenapa `ON` juga diulang rapat di awal? Radio ini half-duplex. Saat robot membalas `ON` dalam hitungan mikrodetik, Pengendali Utama baru saja selesai mengirim dan belum tentu sudah pindah ke mode dengar — `ON` pertama karena itu gampang terlewat. Karena ulangan berikutnya baru datang 1 detik kemudian, tepat sama dengan batas tunggunya, perintah kerap dianggap gagal lalu dikirim ulang dan memboroskan 1 detik. Tiga ulangan berjarak 120 ms di awal gerakan menutup celah itu; irama 1 detik sesuai dokumen spesifikasi tetap berlaku sesudahnya.

---

7. Failsafe & Pelayanan Radio — `serviceRadio()`

Dipanggil terus-menerus **di dalam** loop gerakan. Tiga tugasnya:

1. **Konsumsi semua paket masuk** dan segarkan `lastSignalMs`.
2. **Layani** `STOP` kapan pun, termasuk di tengah gerakan.
3. **Kirim** `ON` **berkala** lewat `maybeSendOn()`.

```
bool serviceRadio()
{
  while (radio.available())
  {
    radio.read(buf, sizeof(buf));
    lastSignalMs = millis();
    if (strncmp(buf, "STOP", 4) == 0) { stopRequested = true; continue; }
    maybeSendTelemetry();      // giliran robot bicara
  }
  maybeSendOn();
  return (millis() - lastSignalMs) < SIGNAL_TIMEOUT_MS;
}
```

Perintah gerak lain yang masuk saat robot sibuk **sengaja diabaikan** — itu adalah paket kiriman ulang dari Pengendali Utama yang belum sempat menerima `ON`. Kalau diproses, satu perintah bisa jadi beberapa gerakan.

Telemetri MPU sengaja diselipkan **hanya setelah ada paket masuk**, bukan tiap iterasi loop, supaya tidak bertabrakan dengan paket perintah yang sedang dikirim Pengendali Utama.

`checkAbort()` membungkusnya menjadi pemeriksaan tunggal yang mengembalikan `MOVE_LOST_SIGNAL`, `MOVE_STOPPED`, atau `MOVE_DONE`.

---

8. Perintah `FW` — `executeForward(distCm)`

Maju lurus sejauh `distCm` sentimeter. Durasi diturunkan dari jarak dan kecepatan jelajah:

```
durationMs = (distCm / 100.0) / ROBOT_SPEED_MPS * 1000
```

Arah hadap saat perintah diterima **dikunci** sebagai `targetYaw`, lalu dipertahankan dengan kendali proporsional:

```
err = targetYaw - yaw            // dinormalisasi ke -180..180
if (mpuOk && |err| > HEADING_DEADBAND)
    correction = clamp(KP_HEADING * err, ±MAX_CORRECTION)

escKiri  = constrain(CRUISE_PWM + correction, 1000, 2000)
escKanan = constrain(CRUISE_PWM - correction, 1000, 2000)
```

Semakin besar simpangan arah, semakin kuat koreksinya — jauh lebih halus daripada koreksi tetap. *Deadband* 2° mencegah motor terus-menerus dikoreksi karena derau sensor.

Karena tiap perintah `FW` mengunci ulang `targetYaw` dari pembacaan terbaru, akumulasi *drift* giroskop tidak menumpuk lintas perintah.

Syarat `mpuOk` pada baris koreksi penting: bila MPU tidak terbaca, `err` dihitung dari sudut palsu yang beku, sehingga koreksi justru **membelokkan** robot secara konsisten ke satu arah. Lebih baik maju lurus tanpa koreksi (lihat bagian 4b).

---

9. Perintah `CW` / `CCW` — `executeTurn(deg, clockwise)`

Berputar sejauh `deg` derajat. Berbeda dengan versi lama yang memakai durasi tetap, putaran kini **ditutup dengan umpan balik sensor**:

```
startYaw = yaw saat perintah diterima
loop:
    delta  = yaw - startYaw           // dinormalisasi ke -180..180
    turned = (clockwise ? -delta : delta) * YAW_SIGN
    if (turned >= deg - TURN_TOLERANCE) selesai
```

`turned` selalu dihitung **positif ke arah yang diperintahkan**, sehingga satu rumus melayani kedua arah.

Dua tenggat dihitung di awal:

```
openLoopMs = deg / TURN_RATE_DPS * 1000        // perkiraan lama putaran
limitMs    = openLoopMs * 3 + 3000             // pengaman mode closed-loop
```

`limitMs` hanya pengaman bila robot tertahan sesuatu — bukan penentu sudut. Bila `mpuOk` salah, putaran **tidak** dibiarkan berjalan sampai `limitMs`; ia berhenti di `openLoopMs` sebagai putaran berbasis waktu. Tanpa ini, setiap belokan dengan MPU mati akan berputar ~5× lipat (lihat bagian 4b).

> **Kalau `CW` malah berbelok ke kiri**, tanda sumbu Z MPU terbalik terhadap pemasangan. Ubah `YAW_SIGN` menjadi `-1`; tidak perlu mengubah logika lain.

---

10. Menjalankan Satu Perintah — `runMove()`

Merangkai seluruh siklus untuk satu perintah:

```
void runMove(const char *move, int dist)
{
  currentMove = move;  moveInProgress = true;  stopRequested = false;

  sendOn(currentMove);                      // "ON" pertama SEGERA

  result = (move == "FW") ? executeForward(dist)
                          : executeTurn(dist, move == "CW");

  stopBoat();  moveInProgress = false;

  if (result == MOVE_DONE) sendFin(move, dist);
  else                     /* tidak kirim FIN */ ;
}
```

Gerakan yang tidak tuntas sengaja tidak dilaporkan. Dengan begitu Pengendali Utama tahu perintah gagal dan **tidak melanjutkan** ke perintah berikutnya — sisa antrean otomatis batal.

---

11. Parsing Paket — `parseCommand()`

Memecah `[Moves]:[Dist]` pada karakter `:`, membersihkan spasi, menaikkan huruf, lalu memvalidasi rentang:

Perintah	Rentang sah	Bila di luar
`FW`	10–200 cm	Ditolak, dicetak ke Serial, paket diabaikan

Perintah	Rentang sah	Bila di luar
`CW` / `CCW`	1–180 derajat	Ditolak, dicetak ke Serial, paket diabaikan

Toleran terhadap spasi, jadi `FW : 100` maupun `FW:100` sama-sama diterima. Perintah tak dikenal langsung ditolak dengan petunjuk format.

Validasi ini adalah **lapisan kedua** — aplikasi sudah menolak nilai di luar rentang sebelum mengirim. Firmware tetap memeriksanya sendiri agar aman terhadap paket rusak atau pengirim lain.

---

12. Inisialisasi — `setup()`

Urutan inisialisasi **penting dan disengaja**:

1. **NRF24 lebih dulu** — `begin()`, PA_LOW, 250 KBPS, channel `0x76`, reading pipe 1, `enableDynamicAck()`, `startListening()`.
2. **MPU6050** — `Wire.begin(21, 22)`, `setClock(100 kHz)`, `setTimeOut(50 ms)`, lalu `mpuProbe()` **dulu**; kalibrasi `calcGyroOffsets(true)` hanya dijalankan kalau sensor benar-benar meng-ACK.
3. **ESC** — alokasi 4 timer PWM, `setPeriodHertz(50)`, `attach(pin, 1000, 2000)`, lalu `armESC()`.

Kenapa NRF paling awal? `calcGyroOffsets` bisa menggantung selamanya bila I2C bermasalah, dan `armESC()` menahan 5 detik. Kalau radio diinisialisasi belakangan, `startListening()` baru aktif ~8 detik setelah boot — Pengendali Utama tidak ter-ACK dan melaporkan gagal.

`ESP32PWM::allocateTimer()` **wajib** dipanggil sebelum `attach()`. Tanpa itu PWM ke ESC bisa gagal diam-diam dan baling-baling tidak bergerak sama sekali.

Bila `radio.begin()` gagal, firmware masuk perulangan tak berujung sambil mencetak petunjuk wiring — lebih baik berhenti terang-terangan daripada berjalan tanpa kendali.

MPU gagal ditangani berbeda dari NRF gagal. Tanpa radio robot tidak bisa diperintah sama sekali, jadi berhenti adalah pilihan yang benar. Tanpa MPU robot masih bisa menjalankan perintah secara open-loop, jadi firmware mencetak petunjuk wiring lalu **tetap berjalan** dengan `mpuOk = false`, sambil terus mencoba menyambung ulang tiap 2 detik. Yang penting: kondisi itu dilaporkan, bukan disembunyikan.

---

13. Loop Utama — `loop()`

Jauh lebih ringkas dari versi sebelumnya karena tidak ada lagi algoritma misi:

```
void loop()
{
  mpu.update();

  if (radio.available())
  {
    radio.read(text, sizeof(text));
    lastSignalMs = millis();          // paket APAPUN = koneksi hidup

    if (data.startsWith("PING")) { maybeSendTelemetry(); return; }

    if (data.startsWith("STOP"))
    {
      stopBoat();  stopRequested = false;
      sendFin("STOP", 0);
      return;
    }

    if (parseCommand(data, move, &dist)) runMove(move, dist);
    else  Serial.println("Format: FW:100 | CW:90 | CCW:90 | STOP:0");
```

```
  }
}
```

Tiga jenis paket dilayani berurutan menurut prioritas: **PING** (heartbeat murni), **STOP** (berhenti darurat, dilayani lebih dulu dari apa pun), lalu **perintah gerak**.

Perhatikan `runMove()` bersifat **blocking** — selama gerakan berlangsung `loop()` tidak kembali. Pelayanan radio tetap berjalan karena `serviceRadio()` dipanggil dari dalam loop gerakan.

---

14. Protokol Paket

Pi → **Robot** (alamat `00001`, channel `0x76`, 250 KBPS, PA_LOW):

Paket	Makna
`FW:<10-200>`	Maju lurus sekian sentimeter
`CW:<1-180>`	Putar searah jarum jam (ke kanan) sekian derajat
`CCW:<1-180>`	Putar berlawanan jarum jam (ke kiri) sekian derajat
`STOP:0`	Hentikan motor — berhenti darurat
`PING`	Heartbeat tiap 0,5 detik (menjaga failsafe + memicu giliran bicara robot)

Robot → **Pi** (alamat `00002`, NOACK):

Paket	Makna
`ON <Moves>`	Perintah sedang dijalankan — dikirim tiap 1 detik selama bergerak
`FIN <Moves>:<Dist>`	Perintah selesai dieksekusi dengan baik
`R=<roll>;P=<pitch>;Y=<yaw>;S=<kode>`	Telemetri sudut MPU (tambahan, di luar spesifikasi wajib). Kode status lihat tabel di bawah

Kode status MPU (`S`) — dikirim lewat radio supaya diagnosa terbaca dari GUI, karena ESP32 yang sudah terpasang di robot umumnya tidak bisa lagi dicolok kabel Serial:

	Arti	Yang harus diperiksa
`1`	Sehat	—
`0`	Bus sehat (pull-up ada) tapi tak ada yang menjawab	SDA & SCL kemungkinan **tertukar**, atau chip MPU rusak
`5`	SDA & SCL diam di **LOW** (tidak ada pull-up)	Modul tidak bertenaga: cek 3.3 V, **GND**, dan solderan kabel SDA/SCL
`2`	Perangkat ada di `0x69`, bukan `0x68`	Pin `AD0` modul ter-HIGH — sambungkan `AD0` ke `GND`
`3`	Terbaca, tapi gyro belum terkalibrasi	Yaw merayap; diamkan robot, firmware mengkalibrasi sendiri
`4`	`0x68` meng-ACK tapi data tidak keluar	Kabel I2C goyang/terlalu panjang, atau catu 3.3 V drop

Siklus lengkap satu perintah:

```
Pi  ->  FW:200
Pi  <-  ON FW             (< 1 detik, kalau tidak -> Pi kirim ulang)
Pi  <-  ON FW             (berulang tiap 1 detik selama bergerak)
Pi  <-  FIN FW:200        (selesai -> Pi lanjut ke perintah berikutnya)
```

Parameter radio (**alamat, channel `0x76`, 250 KBPS, PA_LOW**) **harus identik** dengan sisi Pengendali Utama; kalau berbeda, kedua modul tidak akan terhubung.

---

15. Panduan Kalibrasi (Ringkas)

Yang dikalibrasi	Cara	Gejala bila salah
`ROBOT_SPEED_MPS`	Perintahkan `FW:200` di air, ukur jarak tempuh nyata, sesuaikan nilainya	Jarak tempuh meleset konsisten (selalu kurang / selalu lebih)
`YAW_SIGN`	Perintahkan `CW:90`, amati arah putaran	Robot berputar ke arah **berlawanan** dari perintah
`KP_HEADING`	Amati kelurusan saat `FW` panjang	Terlalu kecil: melenceng. Terlalu besar: bergoyang kiri-kanan
`CRUISE_PWM`	Sesuaikan dengan daya motor & beban	Robot terlalu lambat / terlalu kencang untuk dikendalikan
`TURN_RATE_DPS`	Ukur berapa lama putaran 90°	Hanya memengaruhi batas waktu pengaman, bukan ketepatan sudut

Kalibrasi `ROBOT_SPEED_MPS` **wajib dilakukan di air**, bukan di darat — hambatan air sangat berbeda. Nilai ini juga dipakai aplikasi untuk menghitung perkiraan waktu misi.

---

16. Keterbatasan & Iterasi Berikutnya

Sudah teratasi pada iterasi ini:

- Belok berbasis durasi → sekarang **closed-loop yaw MPU**, `TURN_90_MS` tidak diperlukan lagi.
- Pola jalur tertanam di firmware → sekarang **dikemudikan aplikasi**, bisa diubah tanpa flash ulang.
- Tidak ada konfirmasi eksekusi → sekarang ada `ON` **/** `FIN` dengan pengiriman ulang otomatis.

Masih terbuka:

- **Jarak masih open-loop.** `FW` diturunkan dari waktu tempuh, jadi rentan terhadap arus air dan perubahan beban. Perlu **odometri** atau GPS.
- **Tidak ada sensor RPM/encoder**, sehingga gauge kecepatan di aplikasi tetap berupa estimasi.
- **Putaran menggeser posisi** karena ESC unidirectional. Solusi sebenarnya adalah ESC bidirectional agar bisa berputar di tempat.
- **Belum ada perintah kontrol komponen individual** (mematikan satu motor, mengatur ulang MPU) pada tabel protokol.
- **Isi sinyal balik masih terbatas** — bisa diperluas dengan tegangan baterai, throttle nyata, dan status komponen.

---

17. Ringkasan

Firmware berperan sebagai **eksekutor perintah yang jujur**: menjalankan satu gerakan, melapor bahwa ia sedang berjalan, dan melapor lagi saat selesai — atau diam saja bila gagal, sehingga Pengendali Utama tahu harus berhenti. Pemisahan tanggung jawab ini membuat pola jalur bisa dikembangkan sepenuhnya di sisi aplikasi, sementara firmware tetap kecil, mudah diuji, dan mudah dikalibrasi.